SKALA AI MINI-PROJECT · 과제 1 — BASE AI AGENT 개선

장애유형을 "증상"으로 이해하는 인시던트 대응 에이전트

RAG 분류 · 비동기 동시성 · 과잉 조사 차단 · 수치 근거 검증 — 같은 입력으로 Before/After 실측

Multi-Agent MCP RAG 의존성 주입 LangSmith 모듈화 단위 테스트 24

광주 1반 · G007 · 나용성 · 2026.10

1 주제

운영 장애 질문에 진단·대응 절차를 답하는 에이전트

route → plan → gate → research ×N → replan_check → analysis → write → validate → finalize

무엇을 하나

"auth-service DB가 죽은 것 같아요" 같은 질문을 받으면
① MCP Tool로 **모니터링 지표**를 조회하고
② 장애유형을 정해 MCP Resource로 **대응 매뉴얼(runbook)**을 읽고
③ 지표·매뉴얼만 근거로 **진단과 대응 절차**를 답한다.

FastAPI + LangGraph, SSE로 진행 단계를 실시간 표시한다.

원본(baseline)에서 찾은 약점

분류	알림 키워드 순서로 장애유형 결정 — **사용자 증상 무시**	**정확도 42.1%**
동시성	async 안에서 동기 HTTP + MCP 전역 Lock	**동시 5건 = 1건×5**
계획	언급 안 된 시스템까지 조사 → 알림 혼입	**최대 6개 조회**
검증	"30자 + 1) 포함"만 확인	**수치 날조 통과**

개선 5가지 + 추가 3가지

개선	무엇을	결과
① RAG 분류	키워드 규칙 → runbook 임베딩 의미 검색 (+폴백)	**42% → 100%**
② 동시성	AsyncClient · MCP Lock 제거 · 실제 토큰 스트리밍	**처리량 3.6배**
③ Plan 보정	언급 안 된 시스템 조사를 코드로 차단	**1.53 → 1.16개**
④ 수치 검증	초안 숫자가 수집 사실에 없으면 반려·재작성	LLM 비용 0
⑤ 모듈화	`graph.py` → `nodes/` · `rag/`, 백엔드 중복 → `llm_common.py`	배선/구현 분리

+ 평가 하네스

원본·개선판을 **같은 코드로** 측정. holdout 12문항, LLM-judge, TTFT, 동시성

+ 단위 테스트 24개

stub 주입, 1.4초, 네트워크 0. 정규식 버그 1건을 잡아냄

+ 판단 근거 표시

웹 UI에 RAG 유사도·제외한 시스템·반려 사유 노출

원본에 있던 것 / 내가 개선·추가한 것

기술	원본	개선 · 추가	위치
Multi-Agent	Supervisor(route) → Planner가 task를 `research` / `analysis` 에이전트에 배정 → Writer → Validator	research 워커 N개가 **실제로 동시 실행**되게 async화, Planner 출력을 코드로 보정	`agent/nodes/*`
MCP	Tool·Resource·Prompt 3종, stdio, `find_*` 로 서버 탐색	**전역 Lock 제거** — 세션이 요청 id로 응답을 짝지어 동시 호출 가능	`mcp_client.py`
RAG	없음 (키워드 규칙)	**신규** — runbook 34청크 임베딩, 증상 0.75 + 알림 0.25, 폴백	`agent/rag/`
의존성 주입	`SupervisorDeps` + `partial`	retriever 주입 · `create_deps` 팩토리 · `RAG_ENABLED` 스위치 · 테스트 stub	`deps.py` · `server.py`
LangSmith	노드별 `@traceable`	검색 단계를 `run_type="retriever"` 로 추가, 후보 점수 기록	`rag/retriever.py`
모듈화	`graph.py` 한 파일, 프롬프트 복붙 3곳	배선만 남기고 `nodes/` 6 · `rag/` 4 · 공용 `llm_common.py`	`agent/`
단위 테스트	없음	**신규 24개** — 그래프 e2e · 병렬성 · RAG 폴백 3경로 · 회귀	`tests/`
스트리밍	완성된 답을 한 번에 전달	OpenAI `stream=True` → 토큰 단위 SSE 중계	`openai_llm.py`

각 기술의 실제 동작 화면은 다음 장부터

증상을 1순위 근거로 runbook을 검색한다

```
data/runbooks/*.md (6종)
 └ chunk → [제목+대표 증상] 1개
           [제목+절차 한 줄] N개
           metadata.incident_type
     │ text-embedding-3-small
     ▼ (디스크 캐시)
 InMemoryVectorStore · 34청크 · 코사인

score(유형)
  = 0.75 × sim(사용자 증상)
  + 0.25 × sim(언급된 시스템 알림)
 → 1등 유형

score < 0.25 · API 오류
 → 키워드 규칙 폴백 (멈추지 않음)
```

zsh — all_total_vue_student

```
$ .venv/bin/python -m eval.compare_classifier --set holdout
id    subset   expected         keyword          rag              score
h01   aligned  memory-leak      ✓ memory-leak    ✓ memory-leak    0.5047
h02   aligned  cpu-spike        ✓ cpu-spike      ✓ cpu-spike      0.399
h03   aligned  disk-full        ✓ disk-full      ✓ disk-full      0.4883
h04   aligned  high-latency     ✓ high-latency   ✓ high-latency   0.3666
h05   aligned  network-timeout  ✓ network-timeout ✓ network-timeout 0.4187
h06   multi    memory-leak      ✗ high-latency   ✓ memory-leak    0.384
h07   multi    network-timeout  ✗ cpu-spike      ✓ network-timeout 0.4445
h08   symptom  db-down          ✓ db-down        ✓ db-down        0.4935
h09   symptom  disk-full        ✗ high-latency   ✓ disk-full      0.3723
h10   symptom  high-latency     ✗ db-down        ✗ network-timeout 0.3661
h11   symptom  network-timeout  ✗ cpu-spike      ✗ high-latency   0.3705
h12   symptom  memory-leak      ✗ disk-full      ✓ memory-leak    0.4841
  "keyword_accuracy": 0.5,
  "rag_accuracy": 0.8333,
```

실행 캡처 — 튜닝에 쓰지 않은 holdout 12문항, LLM 없이 분류기만 비교. 키워드 규칙 **50.0%** → RAG **83.3%**. h10·h11은 RAG도 틀림(한계 장에서 설명)

에이전트가 무엇을 근거로 판단했는지 이벤트로 남긴다

zsh — all_total_vue_student

```
$ curl -sN -G localhost:8111/chat/stream --data-urlencode "message=auth-service DB 연결이 전부 실패…"
data: {"type": "RouteDecided", "route": "incident"}
data: {"type": "PlanCreated", "task_count": 1, "targets": ["auth-service"], "dropped_targets": ["cache-service", "gateway-serv
ice", "log-service", "payments-service", "web-01"]}
data: {"type": "GateApplied", "removed_analysis_tasks": 0}
data: {"type": "TaskCompleted", "id": "task-0", "source": "monitoring", "target": "auth-service", "retries": 1}
data: {"type": "ReplanCheck", "round": 0, "is_sufficient": false, "gap_task": "task-gap-runbook", "target": "db-down", "metho
d": "rag", "score": 0.4357, "candidates": [{"incident_type": "db-down", "score": 0.4357}, {"incident_type": "network-timeout",
"score": 0.3263}, {"incident_type": "high-latency", "score": 0.2701}]}
data: {"type": "TaskCompleted", "id": "task-gap-runbook", "source": "runbook", "target": "db-down", "retries": 0}
data: {"type": "ReplanCheck", "round": 1, "is_sufficient": true}
data: {"type": "AnalysisSkipped", "reason": "analysis task 0개"}
   … AnswerToken 이벤트 다수(토큰 단위 스트리밍, 표시 생략) …
data: {"type": "DraftWritten", "attempt": 1, "length": 408}
data: {"type": "Validation", "attempt": 1, "verdict": "합격"}
data: {"type": "Finalize", "pipeline": "incident-v2"}
data: {"type": "FinalAnswer", "reply": "1) 모니터링 상태를 확인한 결과, auth-service의 상태가 …"}
data: [DONE]
```

실행 캡처 — 개선판 `/chat/stream` SSE 원본 출력. AnswerToken 이벤트와 각 줄의 `node` 필드, 최종 답변 본문은 생략

1 **PlanCreated** — Planner가 고른 6개 시스템 중 질문에 없는 5개를 `dropped_targets`로 제외

2 **ReplanCheck** — RAG가 `db-down` 선택(0.4357). 상위 3후보 점수까지 전달

3 **TaskCompleted** — MCP Tool(monitoring, 재시도 1회 성공) · MCP Resource(runbook) 조회

curl -sN -G localhost:8111/chat/stream (LLM_BACKEND=openai)

배선과 구현을 나누고, stub 주입만으로 그래프를 테스트한다

```
agent/
├ graph.py       배선만 (노드·분기)
├ deps.py        SupervisorDeps
├ state.py · systems.py
├ llm_common.py 프롬프트·plan 정규화
├ grounding.py  수치 근거 검사
├ nodes/  route plan research
│         replan analysis write
└ rag/    embedder vector_store
          retriever keyword
```

`build_graph(deps)` 가 `partial(fn, deps=deps)` 로 모든 노드에 주입한다. 테스트는 `StubMCP` ·fake LLM· `HashEmbedder` 를 넣은 deps 하나로 그래프 전체를 돌린다.

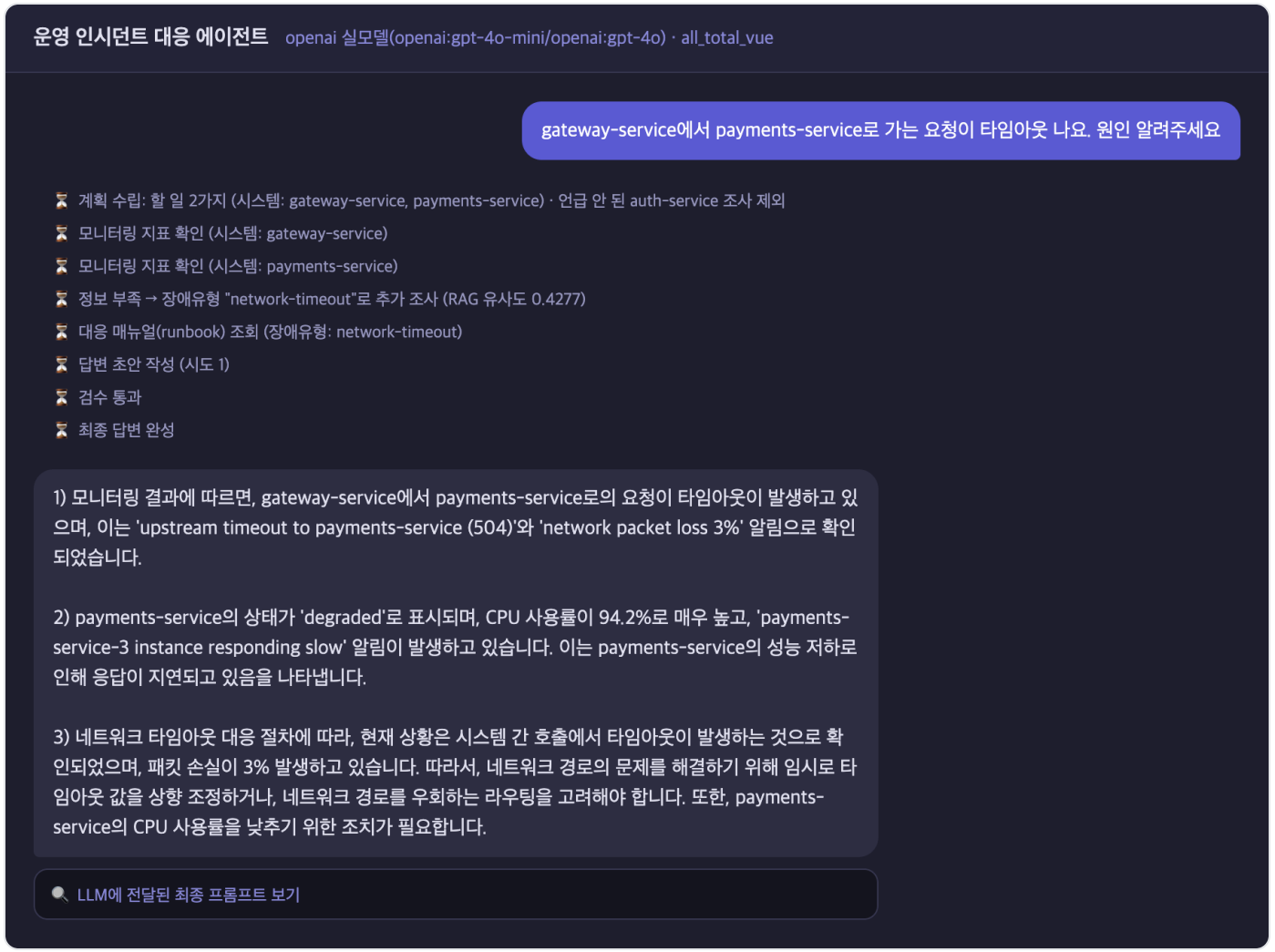
```
zsh — all_total_vue_student

$ .venv/bin/pytest tests -v
tests/test_graph.py::test_gate_removes_self_contradicting_analysis_task PASSED
tests/test_graph.py::test_replan_uses_injected_retriever PASSED
tests/test_graph.py::test_replan_stops_after_one_round PASSED
tests/test_graph.py::test_research_runs_tasks_concurrently PASSED
tests/test_graph.py::test_graph_chat_route_skips_pipeline PASSED
tests/test_graph.py::test_graph_incident_end_to_end_with_rag PASSED
tests/test_rag.py::test_chunk_runbook_has_symptom_and_steps_with_metadata PASSED
tests/test_rag.py::test_vector_store_returns_most_similar_first PASSED
tests/test_rag.py::test_index_covers_every_runbook PASSED
tests/test_rag.py::test_rag_uses_user_symptom_over_alerts PASSED
tests/test_rag.py::test_low_score_falls_back_to_keyword PASSED
tests/test_rag.py::test_embedding_failure_falls_back_to_keyword PASSED
tests/test_rules.py::test_mentioned_systems_in_order_of_appearance PASSED
tests/test_rules.py::test_primary_system_falls_back_to_default PASSED
tests/test_rules.py::test_normalize_plan_drops_unmentioned_systems PASSED
tests/test_rules.py::test_normalize_plan_recovers_when_no_research_task PASSED
tests/test_rules.py::test_normalize_plan_survives_broken_json PASSED
tests/test_rules.py::test_normalize_plan_fixes_unknown_target PASSED
tests/test_rules.py::test_parse_route_fallback_catches_symptom_words PASSED
tests/test_rules.py::test_ungrounded_detects_fabricated_number PASSED
tests/test_rules.py::test_ungrounded_accepts_query_numbers_and_normalized_forms PASSED
tests/test_rules.py::test_check_draft_rejects_format_and_fabrication PASSED
tests/test_rules.py::test_keyword_rule_ignores_user_symptom PASSED
tests/test_rules.py::test_keyword_rule_first_match_wins_on_multi_system PASSED
============================== 24 passed in 1.45s ==============================
```

실행 캡처 — 24개 전부 통과, 1.4초, 외부 네트워크 호출 0

3 최종 실행 화면

웹 채팅 — 복수 시스템 질문



1 **계획 수립** — 질문에 나온 gateway·payments만 조사, auth-service는 제외

2 **모니터링 ×2** — research 워커 2개가 MCP Tool을 동시에 호출

3 **RAG 판정** — network-timeout (유사도 0.4277). 원본은 CPU 알림에 끌려 cpu-spike

4 **검수 통과** — 답변 속 수치가 모두 수집된 지표·알림에 있어 반려 없이 1회 통과

5 **최종 답변** — 토큰 단위로 흘러나오며 1)~3) 형식

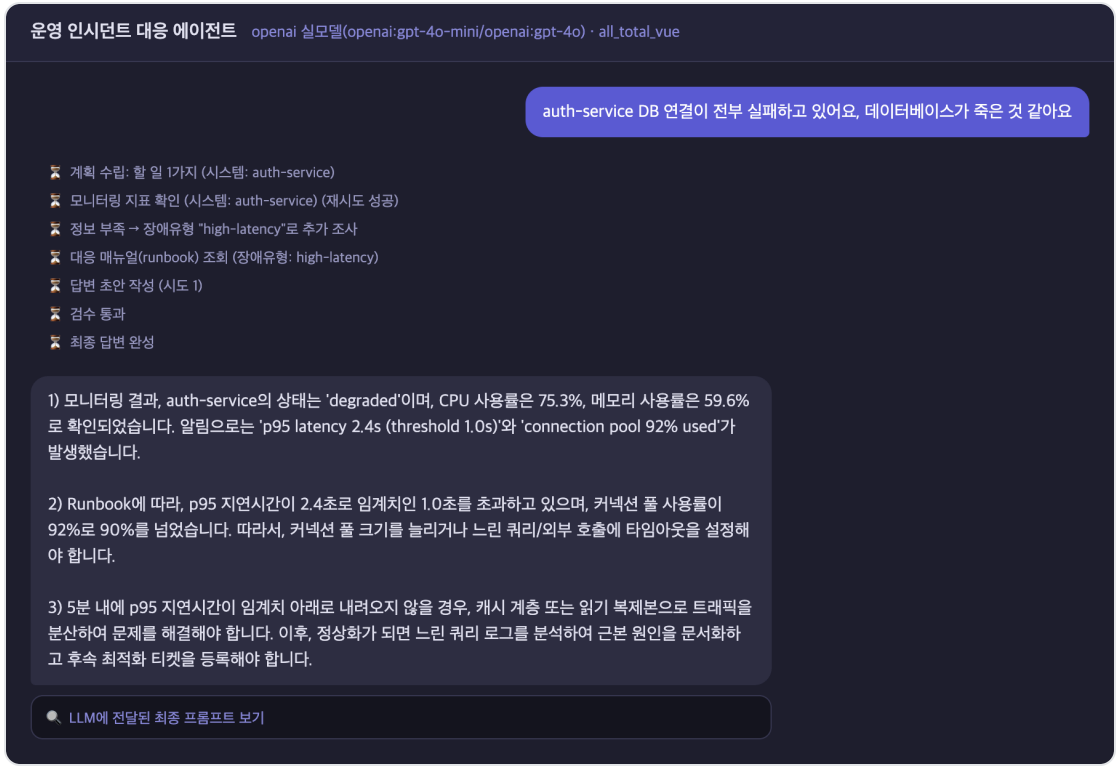
LLM_BACKEND=openai (gpt-4o-mini / gpt-4o) · web_demo_server → server.py

doc/presentation/captures/after_2.png

4 BEFORE / AFTER — 같은 입력

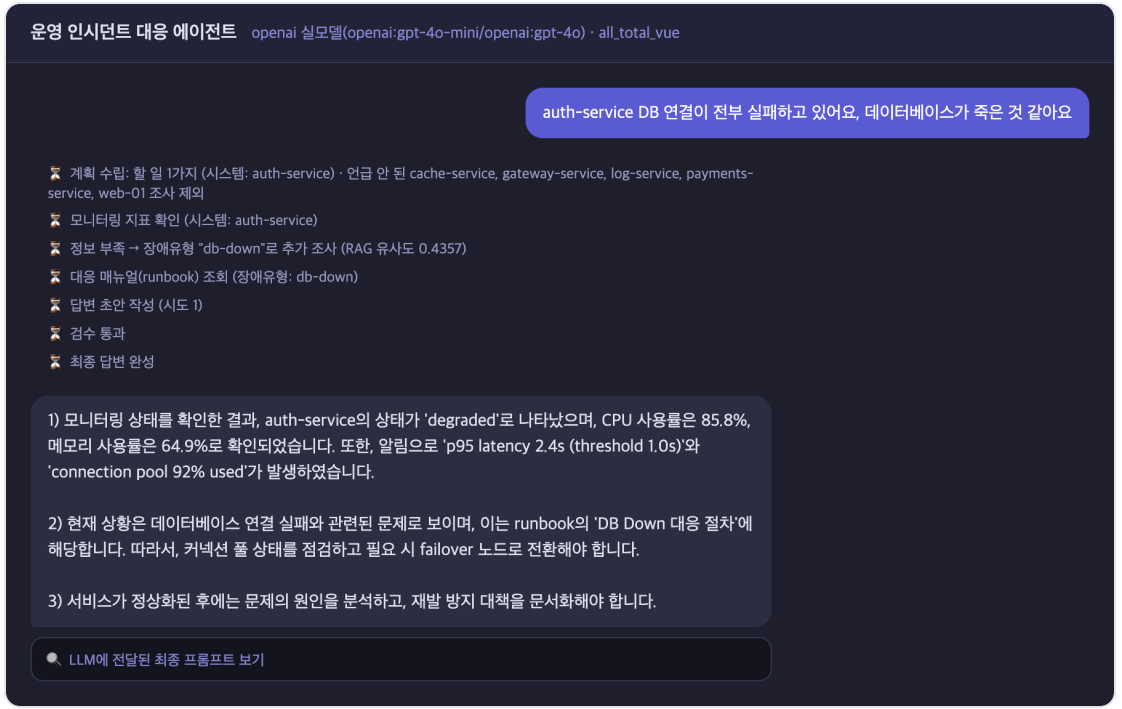
"DB가 죽은 것 같아요" → 원본은 지연 매뉴얼, 개선판은 DB 매뉴얼

Before · baseline 장애유형 high-latency (오답)



알림의 "p95 latency" 키워드가 먼저 걸려 지연 대응 절차(캐시·읽기 복제본)를 안내

After · main 장애유형 db-down (RAG 0.4357)



사용자 증상으로 DB 장애 매뉴얼을 골라 failover 절차를 안내. 언급 안 된 5개 시스템 조사 제외

같은 OpenAI 모델·같은 MCP 서버, 코드만 다름 (captures/before_1.png · after_1.png)

4 BEFORE / AFTER — 수치

정확도 42% → 100%, 동시 처리량 3.6배 — 나빠진 지표까지

zsh — all_total_vue_student

```
$ .venv/bin/python -m eval.report
## 1. End-to-end (LLM_BACKEND=openai, gpt-4o-mini/gpt-4o, eval/dataset.json 22문항)

| 지표 | Before | After | 변화 |
|---|---|---|---|
| incident_type 정확도 (전체 19) | 42.1% | 100.0% | ▲ +57.9%p |
| └ 알림 일치 10 | 80.0% | 100.0% | ▲ +20.0%p |
| └ 복수 시스템 3 | 0.0% | 100.0% | ▲ +100.0%p |
| └ 증상 기반 6 | 0.0% | 100.0% | ▲ +100.0%p |
| route 정확도 | 100.0% | 100.0% | = 0.0%p |
| 요청당 조사 시스템 수 | 1.53 | 1.16 | ▲ -0.37 |
| LLM-judge 진단 적합성(1~5) | 4.42 | 4.89 | ▲ +0.47 |
| LLM-judge 수치 근거(1~5) | 5.00 | 4.74 | ▼ -0.26 |
| 근거 없는 수치 포함 답변 비율 | 0.0% | 0.0% | = 0.0%p |
| 검수(validate) 시도 횟수 평균 | 1.00 | 1.00 | = 0.00 |
| MCP 도구 재시도 횟수 평균 | 0.05 | 0.05 | = 0.00 |
| 단건 지연 평균 | 5.05s | 5.22s | ▼ +0.17s |
| 첫 토큰까지(TTFT) 평균 | 5.49s | 4.68s | ▲ -0.81s |
| 동시 5건 총 소요 | 23.95s | 6.74s | ▲ -17.20s |
| 요청당 비용(가상 단가) | 0.0030$ | 0.0035$ | ▼ +0.0005$ |

동시 처리 처리량: **3.6배** (Before 1건당 4.789s → After 1.349s)
```

실행 캡처 — 평가셋 22문항을 원본·개선판에 같은 하네스로 실행한 결과 집계

83.3%

holdout 분류 정확도 (키워드 50.0%)
평가셋 100%는 편향 가능 → 실성능은 이 값으로 본다

좋아진 것

judge 진단 적합성 4.42→4.89 · TTFT −0.81s · 조사 시스템 수 −0.37

나빠진 · 그대로인 것

단건 지연 +0.17s · 비용 +17% · judge 수치 근거 −0.26
검수 시도·재시도 횟수는 변화 없음 (1.00 / 0.05)

eval/run_eval.py · eval/report.py → doc/results.md

4 BEFORE / AFTER — 동시 요청

같은 질문 5건 동시 전송: 18.38s → 5.31s

Before 단건의 **4.0배** — 한 줄로 서서 처리

```
zsh — all_total_vue_student

$ python -m eval.concurrency_demo http://127.0.0.1:8111 5   # baseline
backend=openai  retriever=(없음 — baseline)
질문: auth-service 로그인 응답이 느려졌어요. 원인 확인하고 대응 방법 알려주세요

단건 순차 3회: 4.59s, 4.09s, 5.02s → 평균 4.57s

동시 5건 전송
 #      시작      완료      소요
 1   0.00s  14.78s  14.78s
 4   0.00s  16.67s  16.67s
 2   0.00s  18.37s  18.37s
 5   0.00s  18.37s  18.37s
 3   0.00s  18.38s  18.37s

총 소요(wall) 18.38s = 단건의 4.0배  (1배에 가까울수록 병렬, 5배에 가까울수록 직렬)
```

After 단건의 **1.1배** — 동시에 처리

```
zsh — all_total_vue_student

$ python -m eval.concurrency_demo http://127.0.0.1:8111 5   # 개선판
backend=openai  retriever=openai:text-embedding-3-small
질문: auth-service 로그인 응답이 느려졌어요. 원인 확인하고 대응 방법 알려주세요

단건 순차 3회: 4.85s, 4.72s, 5.15s → 평균 4.91s

동시 5건 전송
 #      시작      완료      소요
 2   0.00s   5.07s   5.07s
 5   0.00s   5.17s   5.17s
 3   0.00s   5.21s   5.21s
 1   0.00s   5.31s   5.31s
 4   0.00s   5.31s   5.30s

총 소요(wall) 5.31s = 단건의 1.1배  (1배에 가까울수록 병렬, 5배에 가까울수록 직렬)
```

평가 하네스(그래프 직접 호출, 22문항 셋)에서도 23.95s → 6.74s, **3.6배**로 같은 경향이다.

```
# 원인 (baseline)  openai_llm.py: async 노드 안에서 동기 httpx.post → 이벤트 루프 정지
#                   mcp_client.py: async with self._lock → 모든 MCP 호출 직렬화
# 수정             httpx.AsyncClient + await, Lock 제거 (ClientSession이 요청 id로 응답 매칭)
```

eval/concurrency_demo.py — HTTP 서버 전체 경유, 워밍업 1회 제외 · 단건은 순차 3회 평균

개선이 새로 만든 비용과 약점

지연 · 비용 ↔ 정확도

임베딩 1회로 **단건 +0.17s**, runbook 증상 섹션만큼 프롬프트가 길어져 **비용 +17%**. 정확도를 위해 감수했다.

평가셋 편향

평가셋과 증상 문구를 같은 사람이 썼다. 의미를 모르는 **해시 임베딩도 100%**가 나와서, 실성능은 holdout **83.3%**로 본다.

RAG 오분류 · 경험값

"느림"과 "시간 초과"가 임베딩 공간에서 가깝다(h10·h11). 가중치 0.75/0.25·임계값 0.25는 **최적화한 값이 아니다**.

검증의 한계

수치 검사는 문자열 매칭이라 2.4s↔2400ms를 놓친다. LLM-judge는 "데이터 없음"이라는 정직한 답을 감점했다(2건).

차단의 부작용

언급 안 된 시스템 차단은 **보고 싶던 연관 시스템까지** 막을 수 있다. 공유 stdio MCP 서버 1개가 다음 병목이다.

다음 단계

점수 차가 작을 때만 LLM이 6개 유형에서 재판정 · LangSmith Dataset으로 실험 비교 · 재시작 제안 전 `interrupt()` 승인